

MASSMORE TECHNICAL GUIDE

คู่มือการใช้งาน

Massmore Halley

BNO085 / BNO086

9-DOF IMU และ Sensor Fusion สำหรับ ESP32
Arduino IDE • PlatformIO • SH-2/SHTP

พื้นที่สำหรับภาพผลิตภัณฑ์

Massmore Halley BNO085/BNO086

จัดทำโดย **Massmore Biz Co., Ltd.**

ไลบรารี Massmore_BNO08x v1.0.1 • ปรับปรุง 1 กันยายน 2569

อ่านค่า Quaternion, Roll/Pitch/Yaw, acceleration, gyroscope, magnetometer และ motion/activity reports ด้วย ESP32 ผ่าน Arduino IDE และ PlatformIO โดยใช้ library Massmore_BNO08x

เหมาะสำหรับ: ผู้เริ่มต้น, Maker, นักศึกษา, Embedded Engineer, robotics, AR/VR, wearable, motion tracking และระบบที่ต้องการ orientation แบบ real time

เวลาทำ Quick Start: ประมาณ 15–30 นาที หากติดตั้ง Arduino IDE หรือ PlatformIO ไว้แล้ว

ตำแหน่งรูปภาพ 01 — Product overview

ใช้ Block: Upload — ภาพโมดูล Massmore Halley BNO085/BNO086 มุมบน เห็นข้อขา, Qwiic connector และขนาด board ชัดเจน

สารบัญ

1. Overview
2. Features
3. Pinout
4. Introduction และ Spec
5. BNO085 vs BNO086
6. Arduino ESP32 Example
7. PlatformIO Example
8. Examples, Calibration และ Tare
9. Troubleshooting
10. Resources: GitHub, Document และ Datasheet

1. Overview

BNO085 และ BNO086 เป็น 9-DOF IMU ที่รวม sensor และ processing สำคัญไว้ใน package เดียว ได้แก่

- 3-axis accelerometer — วัด acceleration และ gravity direction
- 3-axis gyroscope — วัด angular velocity
- 3-axis magnetometer — วัด magnetic field และช่วยหา heading
- ARM Cortex-M0+ MCU ภายใน chip
- CEVA SH-2 MotionEngine firmware สำหรับ sensor fusion

ความต่างจาก IMU ทั่วไปคือ BNO08x ประมวลผล sensor fusion ภายในตัวเอง แล้วส่งผลลัพธ์ที่พร้อมใช้งาน เช่น quaternion, rotation vector, gravity และ linear acceleration ให้ host MCU โดยตรง จึงลดภาระ ESP32 และลดเวลาที่ต้องใช้ในการออกแบบ/จูนฟิลเตอร์

งานวิจัยด้าน inertial navigation อธิบายตรงกันว่า การอินทิเกรต gyroscope ให้ผลดีในช่วงสั้นแต่เกิด drift เมื่อเวลาผ่านไป จึงต้องอาศัย accelerometer, magnetometer หรือแบบจำลองอื่นช่วยแก้ค่า การใช้ quaternion ยังเหมาะกับการคำนวณ orientation 3 มิติมากกว่า Euler angles เพราะไม่เจอ singularity แบบ gimbal lock ดูพื้นฐานเพิ่มเติมได้จาก [Kok, Hol และ Schön \(2017\)](#) และ [Sabatini \(2011\)](#)

ไลบรารี Massmore_BNO08x ทำหน้าที่สื่อสาร SH-2/SHTP ให้ทั้งหมด รองรับ I2C, SPI, UART-SHTP และ UART-RVC พร้อมตัวอย่าง 14 ชุดสำหรับ Arduino IDE และ PlatformIO

Tip: ถ้าเพิ่งเริ่ม ให้ใช้ I2C + Rotation Vector ที่ 100 Hz ก่อน อย่าเริ่มด้วย SPI หรือ 1 kHz เพราะการตรวจ wiring และ troubleshooting จะยากกว่า

1.1 Features

Orientation และ Rotation Vector

Output	ข้อมูลที่ใช้	เรตสูงสุด	เหมาะกับงาน
Rotation Vector	Accel + Gyro + Mag	400 Hz	เข็มทิศ, หุ่นยนต์ที่ต้องรู้ heading
Game Rotation Vector	Accel + Gyro	400 Hz	VR, gimbal, งานไกล้มอเตอร์
Geomagnetic Rotation Vector	Accel + Mag	90 Hz	งานประหยัดไฟและเคลื่อนที่ไม่เร็ว
AR/VR Stabilized Rotation Vector	9-axis fusion	400 Hz	ภาพ 3 มิติที่ต้องการความนิ่ง
AR/VR Stabilized Game RV	6-axis fusion	400 Hz	AR/VR ในพื้นที่สนามแม่เหล็กรบกวน

Output	ข้อมูลที่ใช้	เรตสูงสุด	เหมาะกับงาน
Gyro-Integrated Rotation Vector	Gyro integration	1000 Hz	rendering, head tracking, control loop อัตราสูง

อัตรารายงานสูงสุดมาจาก [CEVA BNO08X Datasheet v1.17](#) และเป็นค่าสูงสุดราย report ไม่ได้หมายความว่า จะเปิดทุก report ที่ค่าสูงสุดพร้อมกันได้

Motion data

Output	หน่วยในไลบรารี	เรตสูงสุด
Accelerometer	m/s^2	500 Hz
Gyroscope	rad/s หรือใช้ helper เป็น $^{\circ}/s$	400 Hz
Magnetometer	μT	100 Hz
Linear Acceleration	m/s^2 , ตัด gravity แล้ว	400 Hz
Gravity Vector	m/s^2	400 Hz
Raw Accelerometer/Gyroscope/Magnetometer	counts พร้อม timestamp	ขึ้นกับ report

Activity และ gesture detection

Step Counter และ Step Detector
 Personal Activity Classifier
 Stability Classifier และ Stability Detector
 Tap และ Double Tap
 Shake, Flip, Pickup, Tilt, Pocket และ Circle Detector
 Significant Motion Detector
 Sleep Detector

System features

Dynamic calibration และ Save DCD ลง flash
 Tare เพื่อกำหนดทิศ “หน้า” ใหม่ และ persist ได้
 FRS (Flash Record System) read/write
 Product ID และ factory serial number
 Batching และ Report-on-change

Wake sensor และ sleep/on mode
 Callback และอ่านหลาย report ในรอบเดียว
 ไม่มี external dependency และไม่ใช่ malloc/new

1.2 Pinout

ตำแหน่งรูปภาพ 02 — Pinout

ใช้ Block: Upload — Pinout ของ Massmore Halley BNO08x พร้อมลูกศรชี้ VIN, GND, SDA, SCL, INT, RST, SA0, PS0, PS1, CS และ Qwiic

หน้าที่ของขา

ชื่อบนบอร์ดอาจต่างกันเล็กน้อยตาม revision ให้ยึด silkscreen และ schematic ของโมดูลจริงเป็นหลัก

ชื่อขา	หน้าที่	หมายเหตุ
VIN/VCC	ไฟเลี้ยงโมดูล	แนะนำ 3.3V สำหรับเริ่มต้น
GND	กราวด์	ต้องต่อกราวด์ร่วมกับ ESP32
SDA / H_SDA / MISO / TX	I2C data, SPI MISO หรือ UART TX	หน้าที่ขึ้นกับโหมด
SCL / H_SCL / SCK / RX	I2C clock, SPI clock หรือ UART RX	หน้าที่ขึ้นกับโหมด
SA0 / H_MOSI	เลือก I2C address หรือ SPI MOSI	อ่าน SA0 ตอน reset
CS / H_CSN	SPI chip select	active-low
INT / H_INTN	แจ้ง host ว่ามีข้อมูล	active-low; แนะนำให้ต่อ
RST / NRST	รีเซ็ตเซ็นเซอร์	active-low; แนะนำให้ต่อ
PS0 / WAKE	เลือกโปรโตคอล และใช้ปลุกใน SPI	อ่านโหมดตอน reset
PS1	เลือกโปรโตคอล	อ่านโหมดตอน reset
BOOTN	เข้า bootloader เมื่อ LOW ตอน reset	ปกติดึง HIGH ผ่าน 10 kΩ

การเลือกโหมดด้วย PS1/PS0

PS1	PS0	Host interface
LOW	LOW	I2C
LOW	HIGH	UART-RVC
HIGH	LOW	UART-SHTP

PS1	PS0	Host interface
HIGH	HIGH	SPI

ขา PS1/PS0 ถูกอ่านตอน reset การเปลี่ยนจัมเปอร์ระหว่างทำงานจะไม่เปลี่ยนโหมดจนกว่าจะ reset ใหม่ สำหรับ SPI ขาทั้งสองต้อง HIGH ตั้งมาก่อน reset จน H_INTN ทำงานครั้งแรก จากนั้น PS0 กลายเป็นขา WAKE ตามข้อกำหนดใน [BNO08X Datasheet](#)

I2C address

SA0 ตอน reset	7-bit address
LOW	0x4A
HIGH	0x4B

สำคัญสำหรับ Massmore Halley: หน้าสีนั้คำระบุ address เริ่มต้น 0x4A แต่ macro MASSMORE_BNO08X_I2C_ADDR_DEF ในไลบรารีเวอร์ชัน 1.0.1 มีค่า 0x4B คู่มือนี้จึงใช้ 0x4A แบบ explicit หากไม่พบอุปกรณ์ค่อยลอง 0x4B

แรงดันไฟและ logic level

หน้าสีนั้คำ Massmore ระบุแรงดันโมดูล 3.3–5V แต่ตัวชิป BNO08x เองกำหนด VDD 2.4–3.6V และ VDDIO 1.7–3.6V ความสามารถรับ 5V จึงต้องมาจาก regulator/level shifter บนบอร์ดโมดูล

แนวทางที่ปลอดภัย:

1. ต่อ VIN/VCC กับ 3V3 ของ ESP32
2. ใช้ pull-up ของ SDA/SCL ไปที่ 3.3V
3. ห้ามป้อน 5V เข้าขา logic
4. หากจะใช้ไฟ 5V ให้ตรวจ schematic/revision ของโมดูลก่อน

ทริป: Qwiic ใช้ไฟและ logic 3.3V เป็นหลัก จึงเหมาะกับ ESP32 โดยตรง

2. Introduction

Sensor fusion ทำงานอย่างไร

เซ็นเซอร์ทั้งสามชนิดมีจุดเด่นและจุดอ่อนต่างกัน:

เซ็นเซอร์	จุดเด่น	ข้อจำกัด
Accelerometer	รู้ทิศ gravity และวัดความเร่ง	ไวต่อแรงสั่น/การเคลื่อนที่ และบอกทิศเหนือไม่ได้
Gyroscope	ตอบสนองเร็วและลื่น	อินทิเกรตแล้วเกิด drift สะสม
Magnetometer	อ้างอิงสนามแม่เหล็กสำหรับ heading	ถูกรบกวนจากมอเตอร์ เหล็ก แม่เหล็กและสายกระแสสูง

MotionEngine รวมข้อมูลเหล่านี้เพื่อให้ได้ orientation ที่เสถียรกว่าการใช้ sensor ใด sensor หนึ่ง เมื่อสภาพแวดล้อมมีสนามแม่เหล็กรบกวน ให้เปลี่ยนจาก Rotation Vector เป็น Game Rotation Vector เพื่อหลีกเลี่ยงการกระโดดของ yaw แลกกับการยอมให้ yaw drift ช้า ๆ

Quaternion กับ Euler angles

เซ็นเซอร์ส่ง orientation ในรูป quaternion (i, j, k, real) เป็นหลัก ไลบรารีมี helper แปลงเป็น Euler angles:

```
massmore_quat_t q = imu.getQuaternion();
massmore_euler_t e = imu.getEulerDeg();

Serial.println(e.roll); // หมุนรอบแกน X
Serial.println(e.pitch); // หมุนรอบแกน Y
Serial.println(e.yaw); // หมุนรอบแกน Z ช่วง -180..180
Serial.println(imu.getHeadingDeg()); // 0..360
```

ทริป: เก็บ/ส่ง/คำนวณภายในด้วย quaternion แล้วแปลงเป็น Euler เฉพาะตอนแสดงผล จะลดปัญหา gimbal lock และการ interpolate มุมข้าม $\pm 180^\circ$

2.1 Spec

ข้อมูลทั่วไปของชิป

รายการ	ค่า
ประเภท	9-DOF Sensor Fusion SiP
เซ็นเซอร์	3-axis accelerometer + gyroscope + magnetometer
Processor ภายใน	32-bit ARM Cortex-M0+

รายการ	ค่า
Firmware	CEVA SH-2 MotionEngine
Package ชิป	28-pin LGA, 5.2 × 3.8 × 1.1 mm
VDD	2.4–3.6 V
VDDIO	1.7–3.6 V
Operating temperature	–40 ถึง +85 °C
I2C	100/400 kbit/s, address 0x4A หรือ 0x4B
SPI	4-wire, Mode 3, สูงสุด 3 MHz
UART-SHTP	3 Mbit/s, 8N1
UART-RVC	115200 8N1, stream 100 Hz

ค่าด้านบนเป็นสเปกชิป ไม่ใช่ขนาด/แรงดันภายนอกของ breakout board โปรดตรวจหน้าสินค้าและ schematic ของโมดูลประกอบด้วย

Performance ของ sensor fusion

Output	เงื่อนไข	ค่าตาม datasheet
Rotation Vector	Dynamic rotation error	3.5°
Rotation Vector	Static rotation error	2.0°
Game Rotation Vector	Dynamic non-heading error	2.5°
Game Rotation Vector	Static non-heading error	1.5°
Game Rotation Vector	Dynamic heading drift	0.5°/min
Geomagnetic RV	Dynamic rotation error	4.5°
Gravity	Static angle error	1.5°
Linear Acceleration	Dynamic accuracy	0.35 m/s ²
Accelerometer	Dynamic accuracy	0.3 m/s ²
Gyroscope	Dynamic accuracy	3.1°/s
Magnetometer	Dynamic accuracy	1.4 µT

CEVA ระบุเพิ่มเติมว่าการทำงานจริง Rotation Vector มักอยู่ราว 5° และ Geomagnetic Rotation Vector ราว 10° เพราะขึ้นกับสภาพสนามแม่เหล็ก จึงควรรออกแบบ tolerance จากค่าการทำงานจริง ไม่ยึดเพียงค่าจำลองในตาราง

Latency

Output	100 Hz	200 Hz
Gyro Rotation Vector	6.6 ms	3.7 ms
Rotation Vector	6.6 ms	3.7 ms
Game Rotation Vector	6.6 ms	3.7 ms

การใช้พลังงานตัวอย่าง — BNO086

วัดด้วย SPI ที่ VDDIO 3.0V และ VDD 3.3V:

Configuration	VDDIO current	VDD current	Power
Idle	0.047 mA	0.01 mA	0.17 mW
9-axis fusion 100 Hz	3.18 mA	7.50 mA	34.29 mW
9-axis fusion 400 Hz	6.55 mA	7.50 mA	44.40 mW
Gyro RV 1000 Hz	6.84 mA	7.50 mA	45.27 mW
Accelerometer 125 Hz	0.98 mA	0.15 mA	3.44 mW
Step counter 31.25 Hz	0.36 mA	0.14 mA	1.54 mW

BNO085 vs BNO086

BNO086 เป็น drop-in replacement ของ BNO085: pinout, host interfaces และ shared SH-2 reports เหมือนกัน จึงใช้ wiring และ Massmore_BNO08x API ชุดเดียวกันได้ BNO086 เพิ่ม 14-bit accelerometer fusion, ลด idle power และรองรับ Interactive Calibration ตาม [CEVA BNO08X Datasheet](#)

หัวข้อ	BNO085	BNO086	ใช้งานต่างกันอย่างไร
Pinout และ shared reports	รองรับ	รองรับ	ใช้ wiring, begin() และ report API เดียวกัน
Accelerometer fusion	standard resolution	14-bit accelerometer fusion	BNO086 ได้ resolution ใน fusion path เพิ่มขึ้นโดยไม่ต้องเปลี่ยน code

หัวข้อ	BNO085	BNO086	ใช้งานต่างกันอย่างไร
Idle power ตาม CEVA test condition	ประมาณ 0.39 mW	ประมาณ 0.17 mW	BNO086 เหมาะกับระบบที่ idle นาน แต่ต้องวัด power ของ module จริงอีกครั้ง
Dynamic Calibration	รองรับ	รองรับ	ใช้ example 05_Calibration, calibrateAll() และ saveCalibration() เหมือนกัน
Interactive Calibration	ไม่มี	เพิ่มเข้ามา	Library v1.0.1 ยังไม่มี dedicated method สำหรับ feature นี้
UART-RVC Motion Intent/Request	reserved	รองรับ	ใช้ example 12_UART_RVC แล้วอ่าน motionIntent และ motionRequest

สำหรับ Rotation Vector, Game Rotation Vector, acceleration, gyroscope, magnetometer, Step Counter, Calibration และ Tare ไม่ต้องเขียน code แยกตามรุ่น หาก application ต้องใช้ BNO086-only reports ให้ตรวจ API ของ library ก่อน เพราะ version 1.0.1 มี protocol IDs สำหรับ Motion Request, Optical Flow และ Dead Reckoning Pose แต่ยังไม่มี typed high-level enable/get API สำหรับ Optical Flow และ Dead Reckoning Pose

ตำแหน่งรูปภาพ 03 — BNO085/BNO086 comparison

ใช้ Block: Upload — ภาพ BNO085 และ BNO086 วางคู่กัน พร้อม callout “Same wiring/API” และ “BNO086: 14-bit fusion, lower idle power, Interactive Calibration”

3. User Guides Step by step

อุปกรณ์ที่ต้องเตรียม

รายการ	จำนวน	หมายเหตุ
Massmore Halley BNO085/BNO086	1	เลือกรุ่นตามงาน
ESP32 development board	1	ตัวอย่างใช้ ESP32 Dev Module
สาย USB data	1	สายชาร์จออย่างเดียวอัปโหลดไม่ได้
สายจัมเปอร์	6	VIN, GND, SDA, SCL, INT, RST
Breadboard	1	ไม่บังคับ

ตำแหน่งรูปภาพ 04 — Hardware preparation

ใช้ Block: Upload — ภาพ Massmore Halley, ESP32, USB cable และ jumper/Qwiic cable ก่อนเริ่ม wiring

ต่อวงจร I2C

ตำแหน่งรูปภาพ 05 — I2C wiring

ใช้ Block: Upload — Wiring diagram: Massmore Halley BNO08x กับ ESP32 Dev Module พร้อม label GPIO 21, 22, 4 และ 5

BNO08x	ESP32	จำเป็น
VIN/VCC	3V3	✓
GND	GND	✓
SDA	GPIO 21	✓
SCL	GPIO 22	✓
INT	GPIO 4	แนะนำ
RST	GPIO 5	แนะนำ

ตรวจอีกครั้งก่อนเสียบ USB:

- ☐ VIN ต่อ 3V3 ไม่ใช่ 5V

- GND ต่อถึงกัน
- SDA/SCL ไม่สลับ
- เลขพินในโค้ดตรงกับ GPIO จริง
- ไม่มีสายหลวม/เส้นทองแดงแตะกัน

ทริบ: ถ้าใช้ Qwiic ตรวจสอบลำดับสีสายจาก pinout ของสาย/บอร์ด อย่าเชื่อสีอย่างเดียว เพราะสาย third-party บางชุดจัดสีไม่เหมือนมาตรฐาน

3.1 Arduino ESP32 — Example

Step 1 — ติดตั้ง Arduino IDE

ดาวน์โหลด Arduino IDE 2 จาก [Arduino Software](#) แล้วติดตั้งตามระบบปฏิบัติการ หากต้องการรายละเอียดแบบเป็นทางการดู [Download and install Arduino IDE](#)

Step 2 — ติดตั้ง ESP32 board package

1. เปิด **File > Preferences** บน Windows/Linux หรือ **Arduino IDE > Settings** บน macOS
2. ใส่ URL ต่อไปนี้ใน **Additional Boards Manager URLs**

https://espressif.github.io/arduino-esp32/package_esp32_index.json

1. เปิด **Tools > Board > Boards Manager...**
2. ค้นหา esp32
3. ติดตั้ง esp32 by Espressif Systems
4. ปิด/เปิด Arduino IDE ใหม่

อ้างอิงขั้นตอนปัจจุบันจาก [Espressif Arduino-ESP32 installation guide](#)

Step 3 — ติดตั้งไลบรารี Massmore_BNO08x

วิธี Add ZIP:

1. ดาวน์โหลดรีโพสิทอรีเป็น ZIP
2. แยก ZIP หลัก
3. บีบอัดเฉพาะโฟลเดอร์ ArduinoIDE/ เป็น ZIP ใหม่
4. ตรวจสอบใน ZIP ว่ามี library.properties, src/ และ examples/ ที่ระดับบน
5. เปิด **Sketch > Include Library > Add .ZIP Library...**
6. เลือก ZIP ที่สร้างในข้อ 3

7. รวบรวมความติดตั้งสำเร็จ

Arduino รองรับทั้ง Add ZIP และการวางไลบรารีใน sketchbook ตาม [Arduino Help Center](#)

ตำแหน่งรูปภาพ 06 — Arduino IDE library installation

ใช้ Block: Upload — Screenshot เมนู Add .ZIP Library และตำแหน่ง File > Examples > Massmore_BNO08x

Step 4 — เปิดตัวอย่างแรก

เปิด **File > Examples > Massmore_BNO08x > 01_BasicRotationVector**

แก้ส่วนกำหนดพิน:

```
#define SDA_PIN 21
#define SCL_PIN 22
#define INT_PIN 4
#define RST_PIN 5
```

จากนั้นแก้ address สำหรับ Massmore Halley เป็น 0x4A:

```
if (!imu.begin(0x4A, Wire, INT_PIN, RST_PIN)) {
```

และเริ่ม I2C ที่ 100 kHz:

```
Wire.begin(SDA_PIN, SCL_PIN);
Wire.setClock(100000);
```

Step 5 — เลือก board และ port

1. **Tools > Board > esp32 > ESP32 Dev Module** หรือรุ่นที่ตรงกับบอร์ด
2. **Tools > Port >** เลือกพอร์ตที่เพิ่มขึ้นหลังเสียบ USB
3. ตั้ง Upload Speed ตามที่บอร์ดรองรับ เริ่มที่ 460800 หรือ 921600 หากอัปโหลดไม่เสถียรให้ลดเป็น 115200

Step 6 — Verify, Upload และเปิด Serial Monitor

1. กด **Verify** เพื่อตรวจสอบ compile
2. กด **Upload**
3. หากบอร์ดค้างที่ Connecting... ให้กดปุ่ม BOOT ค้าง แล้วปล่อยเมื่อเริ่มเขียน flash
4. เปิด Serial Monitor
5. ตั้ง baud rate 115200

ผลที่ใช้ยืนยันการทำงานคือ terminal แสดง BNO08x connected และค่า quaternion เปลี่ยนต่อเนื่องเมื่อหมุน board

ตำแหน่งรูปภาพ 07 — ผลทดสอบ Arduino IDE

ใช้ Block: Upload — Screenshot Serial Monitor 115200 baud หลัง Upload สำเร็จ โดยให้เห็นข้อความเชื่อมต่อและค่า quaternion หรือ Roll/Pitch/Yaw

พื้นที่สำหรับผลทดสอบ Arduino IDE: ส่วนนี้เว้นว่างไว้สำหรับใส่ screenshot, serial output และข้อสรุปจาก hardware จริง

Test item	ข้อมูลสำหรับผู้ทดสอบกรอก
วันที่ / ผู้ทดสอบ	
ESP32 board / USB port	
Sensor model	BNO085 / BNO086 (วงกลมรุ่นที่ใช้)
I2C address / pins	0x___ SDA GPIO___ SCL GPIO___ INT GPIO___ RST GPIO___
Firmware / library version	
Build / Upload	PASS / FAIL
Device detection	PASS / FAIL
Quaternion หรือ Roll/Pitch/Yaw	PASS / FAIL
หมายเหตุ	

วางภาพผลทดสอบ / Serial Monitor / Terminal output ในกรอบด้านล่าง

พื้นที่ว่างสำหรับผลทดสอบจริง

Step 7 — อ่านเป็น Roll/Pitch/Yaw

เปิดตัวอย่าง 02_EulerAngles หรือใช้โค้ดฉบับสมบูรณ์นี้:

```
#include <Wire.h>
#include <Massmore_BNO08x.h>

constexpr int SDA_PIN = 21;
constexpr int SCL_PIN = 22;
constexpr int INT_PIN = 4;
constexpr int RST_PIN = 5;
```

```

constexpr uint8_t BNO_ADDR = 0x4A;

MassmoreBNO08x imu;
uint32_t lastPrintMs = 0;

void setup() {
  Serial.begin(115200);
  while (!Serial && millis() < 3000) {}

  Wire.begin(SDA_PIN, SCL_PIN);
  Wire.setClock(100000);

  imu.enableDebug(Serial); // ลบบรรทัดนี้เมื่อใช้งานจริงเพื่อลดข้อความ debug

  if (!imu.begin(BNO_ADDR, Wire, INT_PIN, RST_PIN)) {
    Serial.print("Init failed: ");
    Serial.println(MassmoreBNO08x::statusToString(imu.getLastError()));
    while (true) delay(100);
  }

  if (imu.enableRotationVector(10000) != MASSMORE_OK) {
    Serial.println("Cannot enable Rotation Vector");
    while (true) delay(100);
  }

  Serial.println("roll,pitch,yaw,heading,accuracy");
}

void loop() {
  imu.updateAll(16);

  if (!imu.hasNewReport(MASSMORE_SENSOR_ROTATION_VECTOR)) return;
  if (millis() - lastPrintMs < 100) return;
  lastPrintMs = millis();

  massmore_euler_t e = imu.getEulerDeg();
  massmore_accuracy_t a = imu.getAccuracy(MASSMORE_SENSOR_ROTATION_VECTOR);

  Serial.print(e.roll, 2); Serial.print(',');
  Serial.print(e.pitch, 2); Serial.print(',');
  Serial.print(e.yaw, 2); Serial.print(',');
  Serial.print(imu.getHeadingDeg(), 2); Serial.print(',');
  Serial.println(MassmoreBNO08x::accuracyToString(a));
}

```

Tip: report อาจวิ่ง 100 Hz แต่ไม่จำเป็นต้องพิมพ์ Serial 100 ครั้ง/วินาที อ่านและ process ทุก packet ได้ แต่ลด print rate เหลือ 10 Hz เพื่อไม่ให้ terminal เป็น bottleneck

3.2 PlatformIO — Example

Step 1 — ติดตั้ง VS Code และ PlatformIO IDE

1. ติดตั้ง [Visual Studio Code](#)
2. เปิด Extensions

3. ค้นหา PlatformIO IDE
4. กด Install และรอ initialization เสร็จ
5. Reload VS Code หากมีคำแนะนำ

คู่มือ official: [PlatformIO IDE for VS Code](#)

Step 2 — เปิดโปรเจกต์ที่แนบมา

เลือก **File > Open Folder...** แล้วเปิดโฟลเดอร์:

```
Massmore_BNO08x/PlatformIO/
```

ต้องเปิด PlatformIO/ เป็น root เพื่อให้ VS Code พบ platformio.ini และ lib/Massmore_BNO08x/

Step 3 — เลือก environment

เปิด platformio.ini แล้วตั้งค่า:

```
[platformio]
default_envs = esp32dev
```

ตัวอย่าง environment ที่เตรียมไว้:

บอร์ด	environment
ESP32 DevKit/WROOM-32	esp32dev
ESP32-S3 DevKitC-1	esp32-s3-devkitc-1
ESP32-S2 Saola	esp32-s2-saola-1
ESP32-C3 DevKitM-1	esp32-c3-devkitm-1
ESP32-C6 DevKitC-1	esp32-c6-devkitc-1

Step 4 — ตรวจสอบ src/main.cpp

ตั้งพินและเปลี่ยน address เป็น 0x4A สำหรับ Massmore Halley จากนั้นแนะนำให้ใช้:

```
Wire.setClock(100000);
```

เมื่อทำงานเสถียรแล้วค่อยทดลอง 400 kHz หากต้องการ bandwidth เพิ่ม

Step 5 — Build, Upload, Monitor

กดไอคอน PlatformIO ตามลำดับ:

1. **Build** — ต้องขึ้น SUCCESS

2. **Upload** — ส่ง firmware เข้า ESP32

3. **Monitor** — เปิด serial monitor 115200 baud

หรือใช้ terminal ในแพลตฟอร์ม PlatformIO/:

```
pio run  
pio run -t upload  
pio device monitor -b 115200
```

Step 6 — เปลี่ยนตัวอย่าง

ตัวอย่าง PlatformIO อยู่ใน:

```
PlatformIO/examples/<example-name>/main.cpp
```

คัดลอกตัวอย่างที่ต้องการไปเป็น PlatformIO/src/main.cpp ในสำเนาโปรเจกต์ทำงานของคุณ แล้ว Build ใหม่ ควรมี setup() และ loop() เพียงชุดเดียวใน src/

Tip: โฟลเดอร์ lib/Massmore_BNO08x เป็น project-local library จึงย้าย project ไปอีกเครื่องได้โดยไม่ต้องติดตั้ง library แยกใน global environment

ตำแหน่งรูปภาพ 08 — PlatformIO Build และ Upload

ใช้ Block: Upload — Screenshot PlatformIO แสดง environment ที่เลือก และ terminal ขึ้น SUCCESS หลัง Build/Upload

ตำแหน่งรูปภาพ 09 — ผลทดสอบ PlatformIO

ใช้ Block: Upload — Screenshot PlatformIO Monitor 115200 baud โดยใช้ firmware function เดียวกับ Arduino IDE เพื่อให้เปรียบเทียบผลได้ตรงกัน

พื้นที่สำหรับผลทดสอบ PlatformIO: ส่วนนี้เว้นว่างไว้สำหรับใส่ screenshot, serial output และข้อสรุปจาก hardware จริง

Test item	ข้อมูลสำหรับผู้ทดสอบกรอก
วันที่ / ผู้ทดสอบ	
ESP32 board / USB port	
Sensor model	BNO085 / BNO086 (วงกลมรุ่นที่ใช้)
I2C address / pins	0x___ SDA GPIO___ SCL GPIO___ INT GPIO___ RST GPIO___
Firmware / library version	
Build / Upload	PASS / FAIL
Device detection	PASS / FAIL
Quaternion หรือ Roll/Pitch/Yaw	PASS / FAIL
หมายเหตุ	

วางภาพผลทดสอบ / Serial Monitor / Terminal output ในกรอบด้านล่าง

พื้นที่ว่างสำหรับผลทดสอบจริง

3.3 Examples และการนำไปใช้งานจริง

#	ตัวอย่าง	สิ่งที่จะได้
01	BasicRotationVector	quaternion 9-axis 100 Hz
02	EulerAngles	roll/pitch/yaw/heading
03	AccelGyroMag	motion vectors หลายชนิด

#	ตัวอย่าง	สิ่งที่จะได้
04	ChipIDVerify	Product ID, serial, FRS metadata
05	Calibration	calibration + Save DCD
06	Tare	recenter/persist orientation
07	StepCounterActivity	นับก้าวและจำแนกกิจกรรม
08	TapShakeDetector	gesture reports
09	RawSensorData	raw data สำหรับ logging/custom filter
10	HighRateGyroRV	quaternion อัตราสูง
11	SPI_Interface	SPI Mode 3
12	UART_RVC	heading/acceleration 100 Hz
13	MultiReportAdvanced	callback, batching, sleep/wake
14	FRS_Records	อ่าน/เขียน flash records

เลือก report ให้ตรงงาน

หุ่นยนต์เคลื่อนที่ที่ต้องรู้ทิศ: ใช้ Rotation Vector และ calibrate magnetometer หลังติดตั้งเซ็นเซอร์ในตัวหุ่นยนต์จริง วางให้ห่างมอเตอร์/สายแบตเตอรี่

แขนกลหรือ gimbal ไกล้มอเตอร์: ใช้ Game Rotation Vector เพราะไม่ใช้ magnetometer ค่า yaw จะ drift แต่ไม่กระโดดตามสนามแม่เหล็ก

AR/VR: เริ่มจาก AR/VR Stabilized Game RV หากต้องการ smooth orientation โดยไม่อ้างอิงทิศเหนือ และใช้ Gyro-Integrated RV เมื่อระบบ rendering ต้องการข้อมูลถี่มาก

อุปกรณ์นับก้าว: เปิด Step Counter ที่เรตต่ำ ไม่จำเป็นต้องเปิด fusion 400 Hz ทำให้ประหยัดพลังงานกว่า

Robot Vacuum: UART-RVC ใช้สาย data ทางเดียวและ stream 100 Hz เหมาะกับ heading + acceleration แบบง่าย แต่ใช้ report อื่นและคำสั่ง calibration/tare ผ่าน stream ไม่ได้

ตำแหน่งรูปภาพ 10 — Application example

ใช้ Block: Upload — ภาพตัวอย่างการติดตั้งใน robot, gimbal, wearable หรือ motion controller พร้อมระบุตำแหน่ง sensor และแกน X/Y/Z

3.4 Calibration และ Tare

Calibration ที่ถูกต้อง

ใช้ตัวอย่าง 05_Calibration และทำตาม [CEVA Sensor Calibration Procedure](#):

ตำแหน่งรูปภาพ 11 — Calibration movement

ใช้ Block: Upload — ภาพลำดับการหมุน board รอบ Roll, Pitch และ Yaw รวมถึงท่าทางนิ่งสำหรับ Gyroscope calibration

1. เปิด Serial Monitor 115200 baud
2. พิมพ์ c เพื่อเปิด calibration ของ accel + gyro + mag
3. Magnetometer: หมุนบอร์ดประมาณ 180° แล้วกลับ รอบแกน roll, pitch และ yaw ใช้ประมาณ 2 วินาทีต่อแกน
4. Accelerometer: ถือบอร์ดในทิศทางที่ต่างกัน 4-6 ทิศ ค้างประมาณ 1 วินาทีต่อทิศ
5. Gyroscope: วางบอร์ดนิ่งบนพื้นมั่นคง 2-3 วินาที
6. ดู heading error/accuracy จนดีขึ้น
7. พิมพ์ s เพื่อ Save DCD ลง flash
8. พิมพ์ e หากต้องการปิด dynamic calibration หลังบันทึก

อย่าใช้ท่าเลข 8 เป็นขั้นตอนหลักโดยอัตโนมัติ: แนวทาง “แกว่งเป็นเลข 8” พบได้บ่อยในคู่มือ sensor อื่น แต่คู่มือ BNO08x ของ CEVA ระบุการหมุนไป-กลับรอบแต่ละแกนชัดเจน

สภาพแวดล้อมของ magnetometer

ทำ calibration หลังติดตั้งเซ็นเซอร์ใน enclosure จริง

ปิดหรือจัดตำแหน่งมอเตอร์/สายกระแสนสูงให้เหมือนสภาวะใช้งาน

หลีกเลี่ยงโต๊ะเหล็ก แม่เหล็ก ลำโพง และไขควงแม่เหล็ก

ถ้า heading ยังไม่นิ่ง ให้เปรียบเทียบ Game Rotation Vector เพื่อแยกปัญหาแม่เหล็กออกจากปัญหาการสื่อสาร

งานวิจัยด้าน inertial/magnetic orientation แสดงว่าความผิดปกติของสนามแม่เหล็กมีผลโดยตรงต่อความน่าเชื่อถือของ heading และต้องอาศัยการชดเชย/weighting ที่เหมาะสม ดู [Sabatini 2006](#) และ [Magnetometer calibration using inertial sensors](#)

Tare — กำหนดทิศหน้าใหม่

ทำ calibration ให้เสร็จก่อน จากนั้นเปิด 06_Tare:

- z — ตั้ง yaw ปัจจุบันเป็นศูนย์ เหมาะกับปุ่ม Recenter
- a — ตั้งศูนย์ทุกแกน ต้องวางอุปกรณ์ระดับและหันไปทิศอ้างอิง
- p — บันทึก tare ให้คงอยู่หลังปิดเปิดเครื่อง
- c — ล้าง tare ที่บันทึกไว้

อ่านข้อจำกัดและ basis ของ tare เพิ่มเติมใน [CEVA Tare Function Usage Guide](#)

3.5 Troubleshooting

อาการ 1 — BNO08x not found

ทำตามลำดับนี้:

1. ถอด USB แล้วตรวจ VIN=3.3V, GND, SDA, SCL
2. ตรวจพิน ESP32 ในโค้ด
3. ใช้ address 0x4A สำหรับ Massmore Halley
4. หากยังไม่พบ ลอง 0x4B
5. ลด I2C เป็น 100 kHz
6. ต่อ INT และ RST
7. reset ใหม่หลังเปลี่ยน SA0/PS0/PS1
8. ตรวจ BOOTN ต้องไม่ LOW ตอน reset

I2C scanner แบบสั้น:

```
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  Wire.setClock(100000);

  for (uint8_t addr = 1; addr < 127; ++addr) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {
      Serial.printf("Found I2C device at 0x%02X\n", addr);
    }
  }
}

void loop() {}
```

อาการ 2 — สแกนเจอแต่ begin() ไม่ผ่าน

ตรวจว่าเป็น BNO08x จริง ไม่ใช่ BNO055 ซึ่งใช้โปรโตคอลคนละแบบ

เปิด imu.enableDebug(Serial); ก่อน begin()

ต่อ INT/RST และรอ reset ให้ครบ

หลีกเลี่ยงการ poll เร็วด้วย loop ที่บล็อก bus

ตรวจ pull-up SDA/SCL ไป 3.3V; datasheet แนะนำค่าทั่วไป 2–4 kΩ โดยขึ้นกับ bus capacitance

อาการ 3 — ข้อมูลหยุดเมื่อเพิ่ม report

ลดจำนวน report
 ลด report rate
 เรียก `updateAll()` ให้ถี่
 อย่าใช้ `delay()` ยาว
 ลดการพิมพ์ Serial
 ใช้ SPI สำหรับ bandwidth สูง

อาการ 4 — yaw กระโดดเมื่อมอเตอร์ทำงาน

นี่มักเป็น magnetic interference ไม่ใช่ bug ของ I2C:

1. เปลี่ยนเป็น Game Rotation Vector เพื่อทดสอบ
2. ย้ายเซ็นเซอร์ให้ไกลมอเตอร์ สายเฟส สายแบตเตอรี่ และโครงเหล็ก
3. บิดคู่สายกระแสดังและลด loop area
4. calibrate ใหม่หลังประกอบจริง
5. หากงานต้องการทิศเหนือมาก ให้ประเมินการใช้อ่างอิงภายนอกเพิ่มเติม

อาการ 5 — มุมกลับแกนหรือหมุนผิดทิศ

ตรวจ orientation ของ PCB และแกน X/Y/Z บนรูป pinout
 อย่าสลับ roll/pitch/yaw ด้วยการเดา
 ถ้าติดตั้งหมุนถาวร ให้ใช้ System Orientation FRS หรือแปลง quaternion อย่างเป็นระบบ
 ทดสอบที่ละแกน: วางนิ่ง แล้วหมุน $+90^\circ$ รอบแกนที่รู้แน่นอน

อาการ 6 — PlatformIO compile ไม่พบ header

ตรวจโครงสร้าง:

```
PlatformIO/
├── platformio.ini
├── src/main.cpp
└── lib/Massmore_BNO08x/src/Massmore_BNO08x.h
```

จากนั้นกด **PlatformIO: Rebuild IntelliSense Index** และ Build ใหม่

4. Resources

4.1 GitHub

[Massmore BNO08x GitHub repository](#)

[Massmore legacy BNO0xx examples](#)

[Arduino IDE examples](#)

[PlatformIO project](#)

หากลิงก์รีโพสิทอรีใหม่ยังไม่เผยแพร่ให้ใช้ไฟล์ในแพ็คเกจ Massmore_BNO08x ที่ได้รับมาจนกว่าหน้า GitHub จะเปิดเป็น public

4.2 Document & Datasheet

เอกสารจากผู้ผลิต

[CEVA BNO08X Datasheet — 1000-3927 v1.17](#)

[CEVA SH-2 Reference Manual — 1000-3625](#)

[CEVA BNO08X Sensor Calibration Procedure — 1000-4044](#)

[CEVA BNO08X Tare Function Usage Guide — 1000-4045](#)

[CEVA BNO085 Development Kit Quick Start Guide](#)

เครื่องมือพัฒนา

[Massmore Halley BNO086/BNO085 product page](#)

[Arduino IDE download](#)

[Arduino: Install libraries](#)

[Espressif Arduino-ESP32 installation](#)

[PlatformIO IDE for VS Code](#)

[PlatformIO project configuration](#)

งานวิจัยและบทความวิชาการพื้นฐาน

Kok, Hol และ Schön, [Using Inertial Sensors for Position and Orientation Estimation](#) —

อธิบาย drift, sensor models และวิธี fusion

Sabatini, [Kalman-Filter-Based Orientation Determination Using Inertial/Magnetic Sensors](#) — quaternion EKF และผลของ magnetic disturbance

Sabatini, [Quaternion-based extended Kalman filter for determining orientation by inertial and magnetic sensing](#) — การรวม gyro, accelerometer และ magnetometer

Schön et al., [Magnetometer calibration using inertial sensors](#) — calibration และผลต่อ heading

Madgwick, [An efficient orientation filter for inertial and inertial/magnetic sensor arrays](#) — พื้นฐาน quaternion gradient-descent filter สำหรับเปรียบเทียบแนวคิด

Checklist ก่อนนำไปใช้ในผลิตภัณฑ์จริง

- ☐ ยืนยัน supply และ logic voltage จาก schematic ของ module revision จริง
- ☐ ยืนยัน I2C address หลังประกอบ
- ☐ ต่อ INT/RST เพื่อ recovery ที่เชื่อถือได้
- ☐ ทดสอบที่ 100 kHz ก่อนเพิ่มเป็น 400 kHz
- ☐ เลือก Rotation Vector ให้ตรงกับสภาพสนามแม่เหล็ก
- ☐ ทำ calibration หลังติดตั้งใน enclosure
- ☐ ทดสอบเมื่อมอเตอร์/โหลดกระแสสูงทำงานจริง
- ☐ จำกัดอัตราพิมพ์ log ไม่ให้รบกวนการอ่าน packet
- ☐ ตรวจ behavior หลัง brownout, reset และถอดเสียบไฟซ้ำ
- ☐ บันทึก firmware version, address, pin map และ calibration procedure ในเอกสารผลิต

บทความโดย **Massmore Biz Co., Ltd.** — เว็บไซต์ www.massmore.shop — อัปเดตเนื้อหา 1 กันยายน 2569

5. Engineering Validation Guide

ส่วนนี้ขยายจากบทความบนเว็บไซต์สำหรับผู้ที่ต้องนำ module ไปใช้จริง โดยเน้น test method ที่ทำซ้ำได้ การบันทึก environment และการแยก software issue ออกจาก wiring, power และ magnetic interference

5.1 เปรียบเทียบผล Arduino IDE และ PlatformIO อย่างยุติธรรม

ใช้ ESP32 board, Massmore Halley module, USB cable และ wiring ชุดเดียวกัน

ใช้ source code logic เดียวกัน: address, pins, report type, report interval และ Serial baud ต้องตรงกัน

Power-cycle ก่อนทดสอบแต่ละ environment เพื่อให้ reset state และ calibration state เริ่มจากเงื่อนไขเดียวกัน

เก็บ output อย่างน้อย 30 วินาทีขณะวางนิ่ง แล้วหมุน +90° รอบ Roll, Pitch และ Yaw ที่ละแกน บันทึก build result, upload result, device detection, update rate, reset recovery และข้อความ error โดยไม่สรุปจาก screenshot เพียงภาพเดียว

หากค่าต่างกัน ให้เปรียบเทียบ library source/version และ compiler flags ก่อนเปลี่ยน hardware

Acceptance principle: Arduino IDE และ PlatformIO ไม่จำเป็นต้องพิมพ์ timestamp ตรงกันทุกบรรทัด แต่ต้อง detect sensor รุ่นเดียวกัน, เปิด report เดียวกัน และให้ orientation response ไปในทิศทางเดียวกัน ภายใต้ motion sequence เดียวกัน

5.2 อ่าน Quaternion, Euler angles และ Accuracy ให้ถูกต้อง

Field	ความหมาย	วิธีตรวจ
Quaternion i, j, k, real	Orientation representation ที่ไม่เกิด gimbal lock	Quaternion norm ควรใกล้ 1 และค่าเปลี่ยนต่อเนื่องเมื่อหมุน
Roll / Pitch / Yaw	Euler angles ที่คำนวณจาก quaternion	ตรวจที่ละแกนด้วยการหมุน +90° และจด sign convention
Heading	Yaw ที่อ้างอิง magnetic north เมื่อใช้ 9-axis Rotation Vector	เปรียบเทียบเมื่ออยู่ห่าง motor/steel และหลัง calibration
Accuracy	ค่าประเมินความเชื่อมั่นจาก SH-2	รอจน Medium/High ก่อนบันทึก calibration หรือ tare
Timestamp / report interval	เวลาของ sensor report และ requested update period	วัดจากหลาย packet; อย่าใช้ Serial print rate แทน sensor report rate

ข้อควรระวัง: Euler angle convention, axis direction และ sensor mounting orientation ต้องกำหนดให้ชัดเจนใน product document การสลับ Roll/Pitch/Yaw ให้ถูกต้องโดยการเดาอาจทำให้ค่าถูกเฉพาะบางท่าแต่ผิดเมื่อหมุนหลายแกนพร้อมกัน

5.3 Wiring และ I2C signal validation

วัด VIN/VCC และ 3.3V rail ด้วย multimeter ขณะ idle และขณะ ESP32 ส่งข้อมูลผ่าน Wi-Fi/Bluetooth

ตรวจ SDA/SCL idle HIGH และไม่มี 5V logic ป้อนเข้าขา BNO08x

เริ่มที่ 100 kHz; เพิ่มขึ้นเป็น 400 kHz หลังผ่าน soak test และตรวจ rise time หากมี oscilloscope ใช้สายสั้น, ground reference ที่ดี และหลีกเลี่ยงการเดิน I2C ขนานกับ motor phase หรือ switching node

ทดสอบทั้ง address 0x4A/0x4B เฉพาะเมื่อ wiring SA0 ไม่ชัดเจน และบันทึกค่าที่พบจริงใน test record

ต่อ INT และ RST เพื่อให้ data-ready handling และ recovery ทำงานได้สม่ำเสมอ

5.4 Reset, brownout และ long-run test

Test	Procedure	Pass condition
Cold boot	ถอดไฟอย่างน้อย 5 วินาที แล้วเปิดใหม่ 10 รอบ	Detect sensor และเริ่ม report ได้ทุกครั้ง
MCU reset	กด EN/RESET ของ ESP32 ขณะ sensor ยังมีไฟ	Library reinitializes โดยไม่ต้องถอดไฟ sensor
Sensor reset	สั่ง hard/soft reset ตาม API	Product ID และ report กลับมาภายใน timeout ที่กำหนด
Brownout simulation	ลด supply ภายใต้อุปกรณ์ที่ปลอดภัยหรือสลับ load สูง	ระบบไม่ค้างถาวรและ recovery path ทำงาน
Soak test	อ่าน report ต่อเนื่องอย่างน้อย 30–60 นาที	ไม่มี packet stall, heap issue หรือ Serial bottleneck
EMI / motor test	เปิด motor, relay, DC/DC และ radio ตามสภาวะจริง	ไม่มี bus error; heading behavior ตรงกับ report ที่เลือก

5.5 BNO085/BNO086 deployment notes

Shared reports ใช้ API เดียวกัน จึงควรเก็บ sensor model และ product ID ใน startup log แทนการแยก firmware โดยไม่จำเป็น

14-bit accelerometer fusion และ lower idle power ของ BNO086 เป็น device-level benefit; application code สำหรับ shared reports ไม่ต้องเปลี่ยน

Interactive Calibration เป็น BNO086 capability แต่ library v1.0.1 ไม่มี dedicated high-level method จึงใช้ standard calibration example เป็น workflow หลักจนกว่าจะมี API ที่ชัดเจน UART-RVC บน BNO086 เพิ่ม motionIntent และ motionRequest; ตรวจสอบ field เหล่านี้เฉพาะเมื่อ sensor model และ RVC packet format รองรับ

Optical Flow และ Dead Reckoning Pose IDs ใน protocol definitions ไม่เท่ากับมี production-ready typed API ต้องตรวจสอบ library release notes ก่อนใช้ใน product requirement

5.6 Release record

รายการ	ค่าที่อนุมัติสำหรับ release
Sensor model / module revision	
ESP32 board / firmware commit	
Massmore_BNO08x version	
Interface / address / pins	
Report types / intervals	
Calibration procedure / DCD policy	
Tare policy / mounting orientation	
Arduino IDE test result reference	
PlatformIO test result reference	
ผู้อนุมัติ / วันที่	